

OPERATING SYSTEMS

Unit-2: Processor Management & Deadlock

Comprehensive Study Notes

1. Processor Management

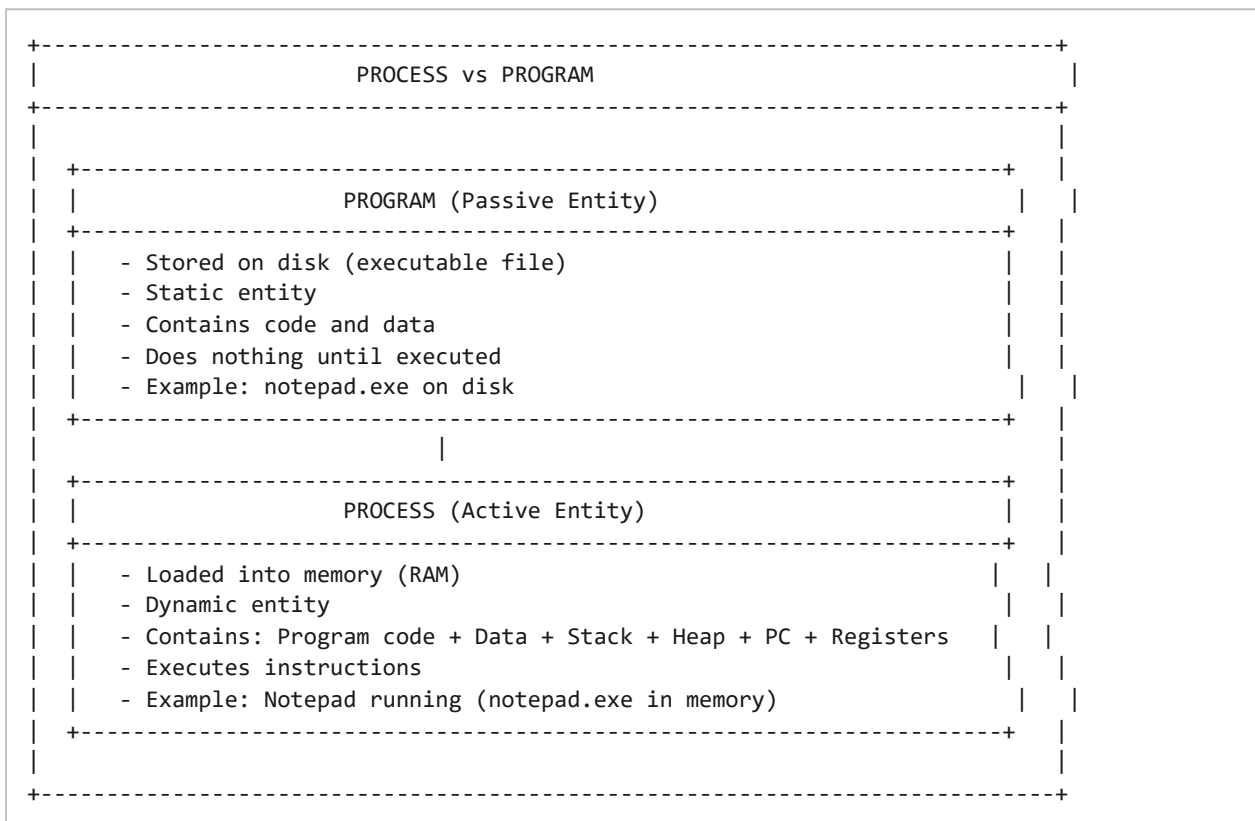
Processor Management is the function of an operating system that manages the **CPU** (Central Processing Unit). It involves:

- **Scheduling** processes for execution
- **Allocating** CPU time to processes
- **Tracking** process states and transitions
- **Managing** process control information

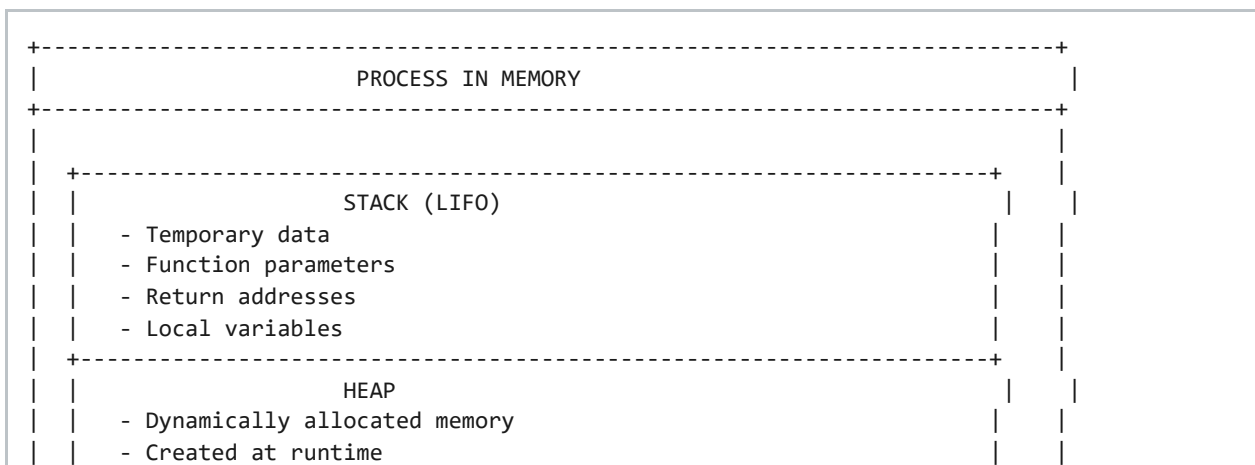
2. Process

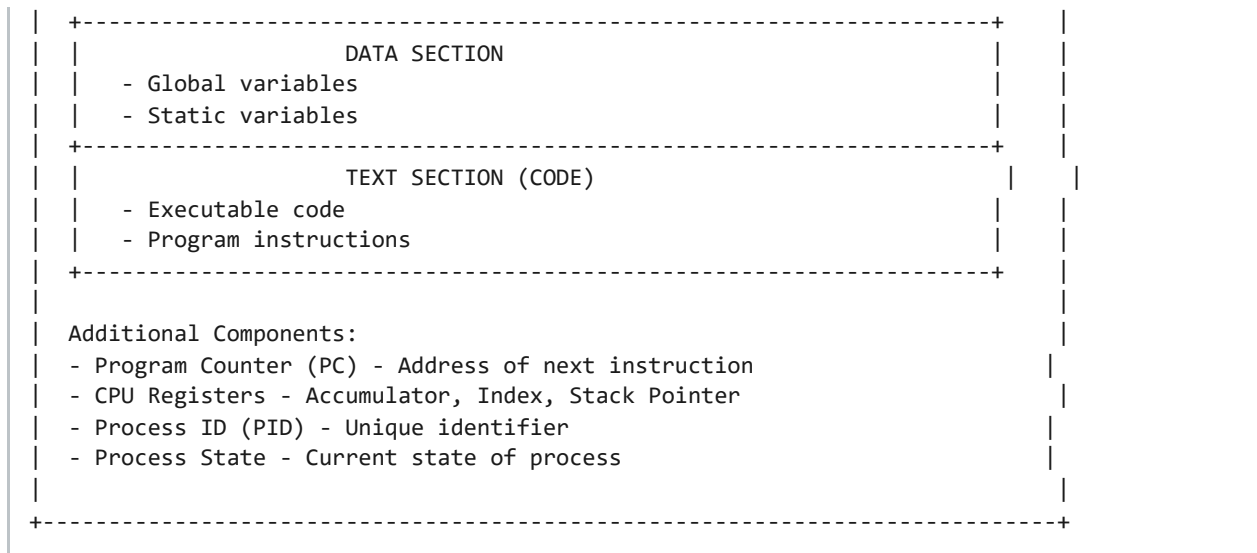
A **Process** is a **program in execution**. It is an **active entity** that represents the execution of a program.

Process vs Program

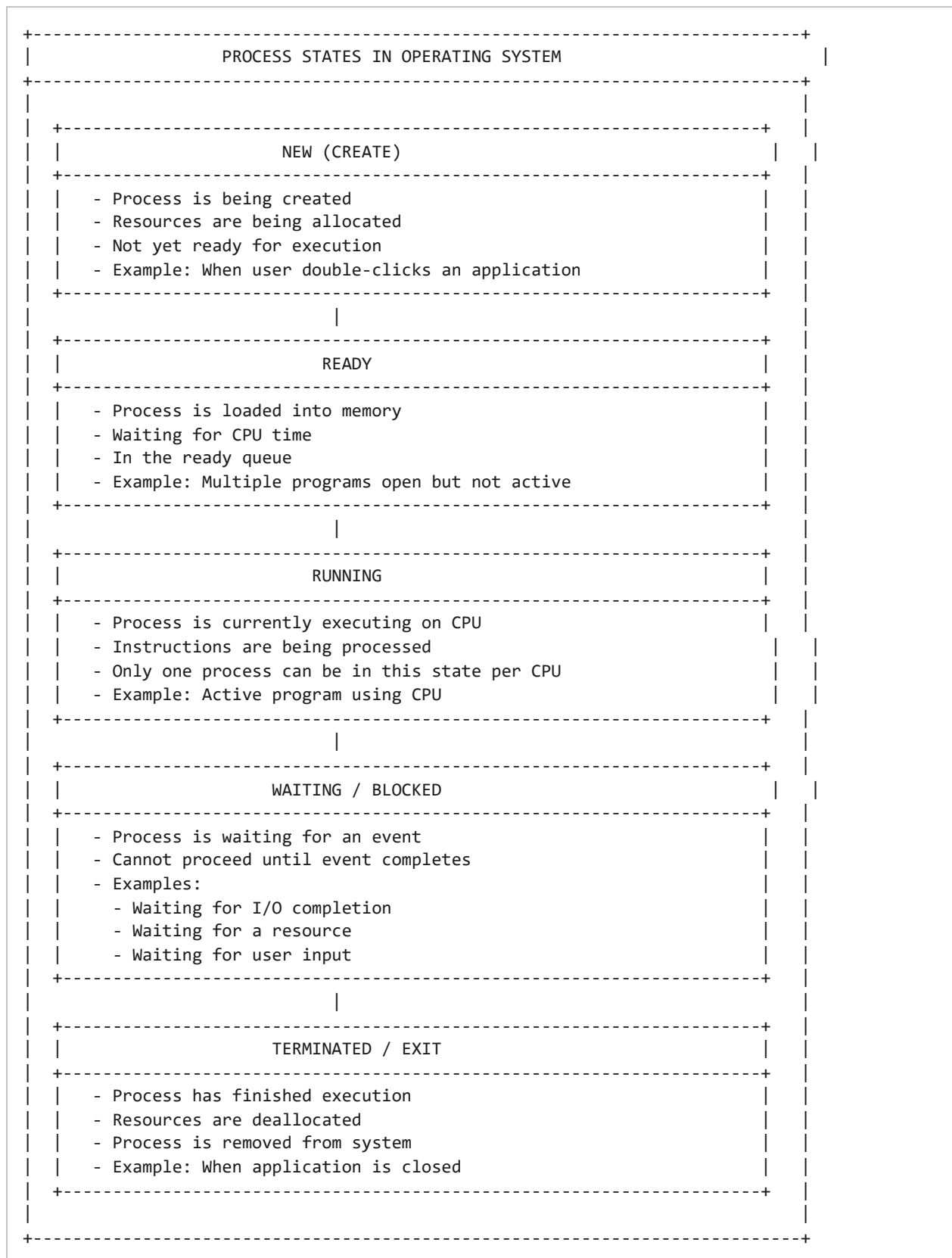


Components of a Process

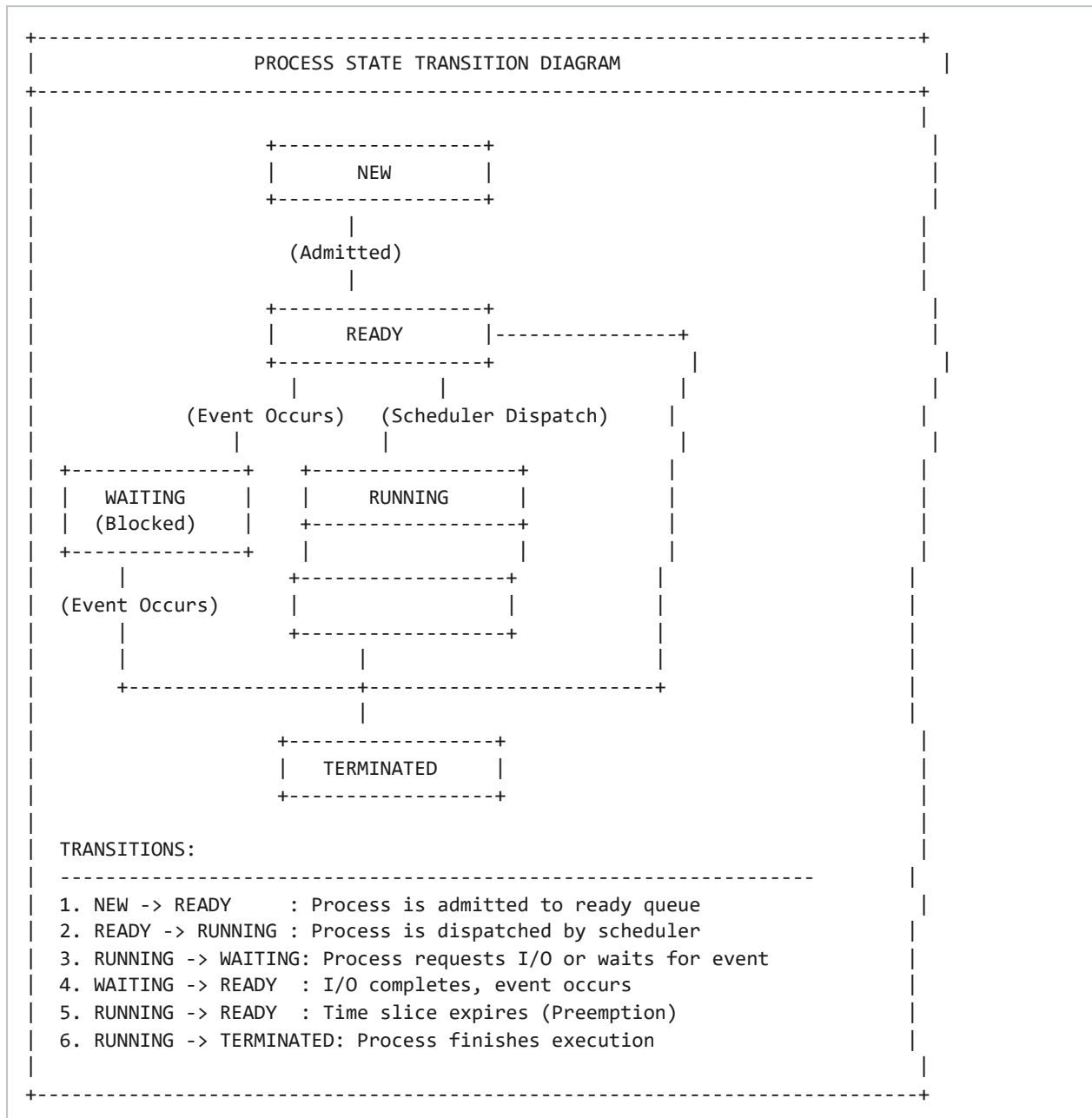




3. Process States



4. Process State Transition Diagram



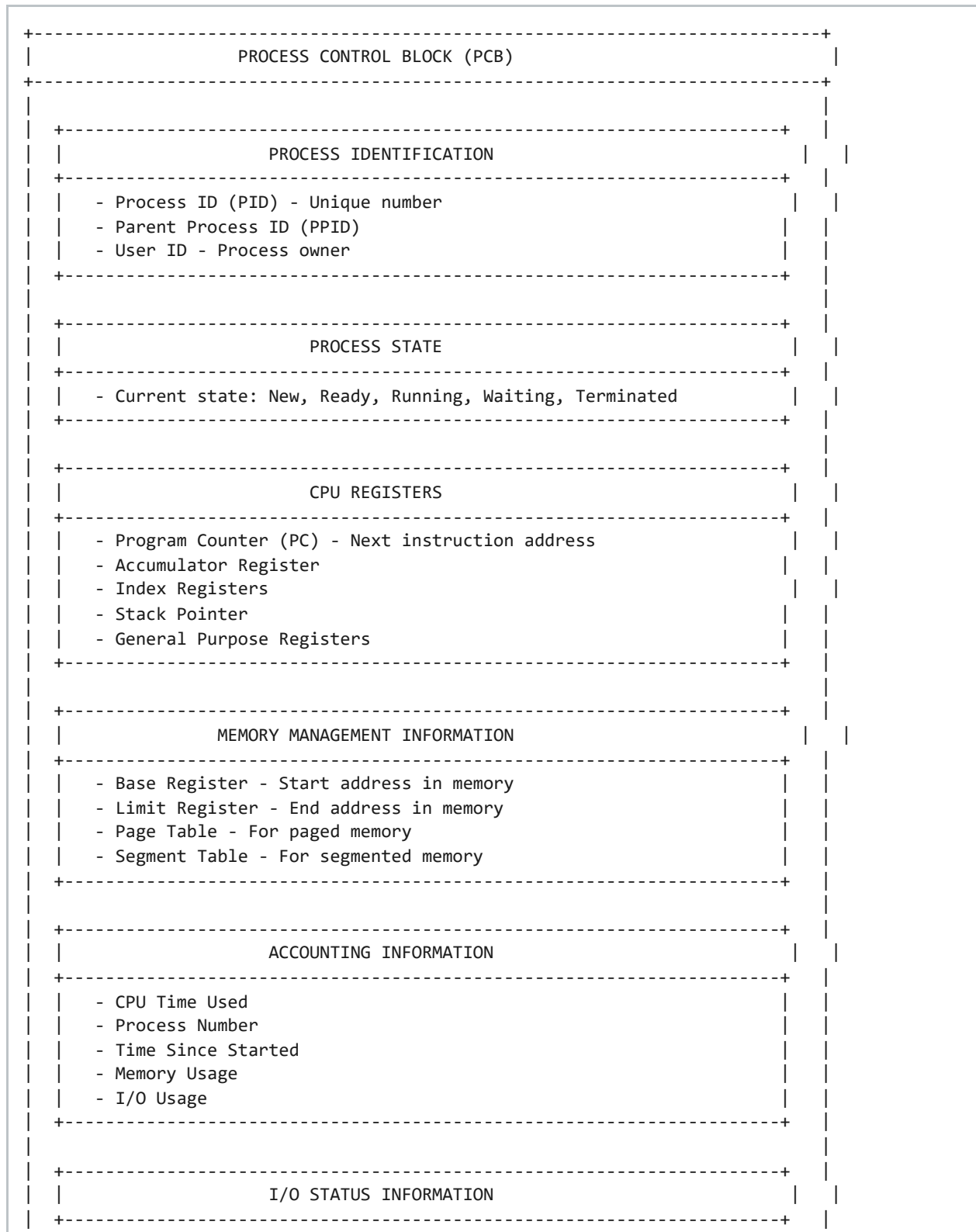
Transition Conditions

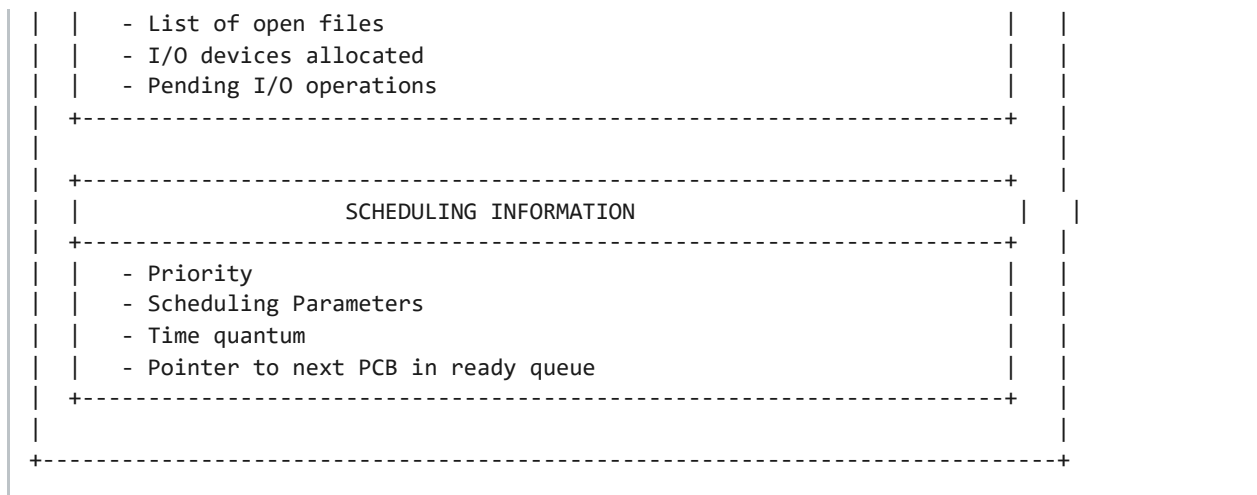
Transition	Condition	Description
New -> Ready	Admission	Process is ready to run
Ready -> Running	Dispatch	Scheduler selects process

Running -> Waiting	I/O Request	Process needs I/O
Waiting -> Ready	I/O Completion	Event has occurred
Running -> Ready	Preemption	Time slice expired
Running -> Terminated	Exit	Process completed
Ready -> Terminated	Abort	Parent terminates child

5. Process Control Block (PCB)

The **Process Control Block (PCB)** is a **data structure** maintained by the operating system for every process. It contains all the information needed to manage a process.



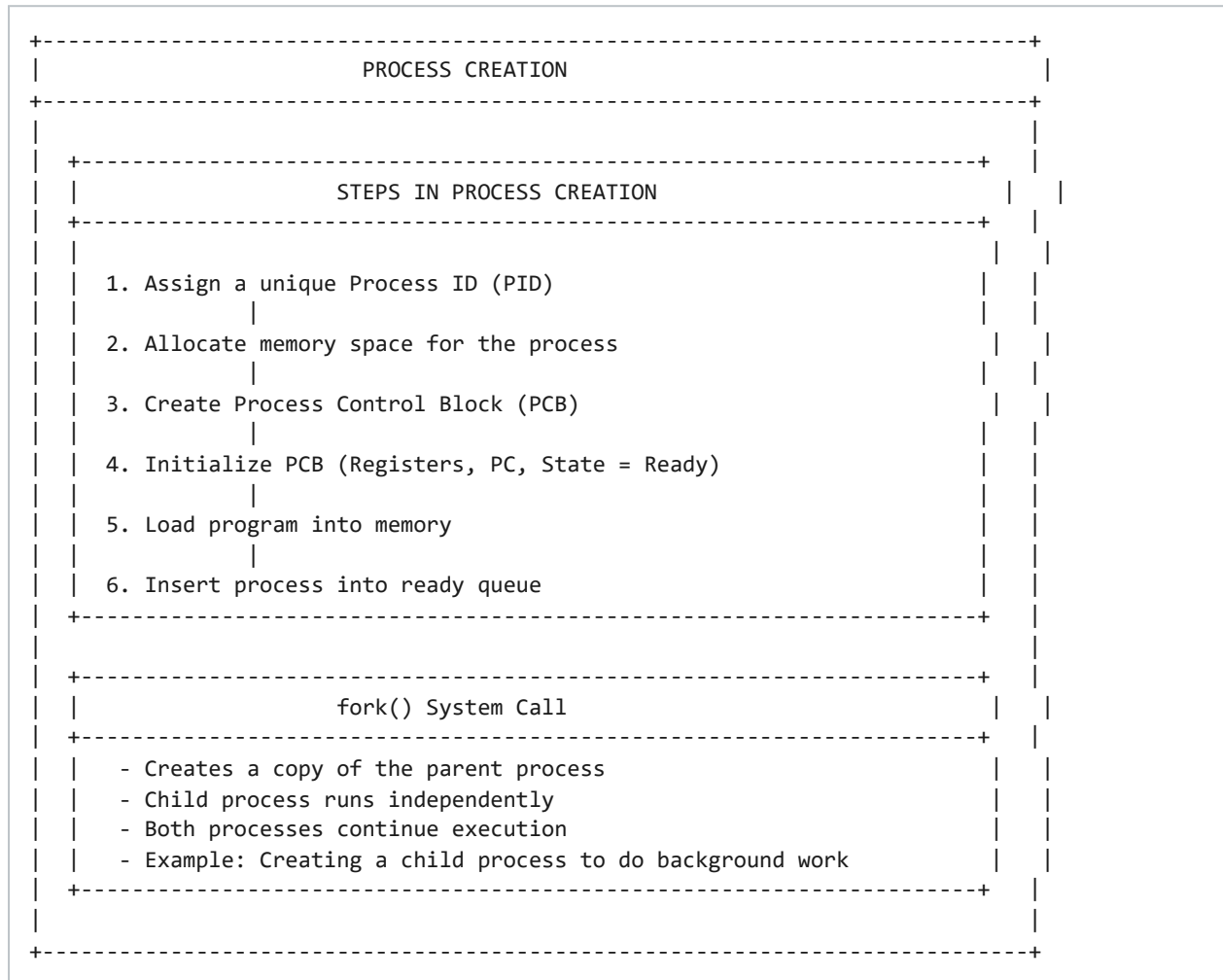


Importance of PCB

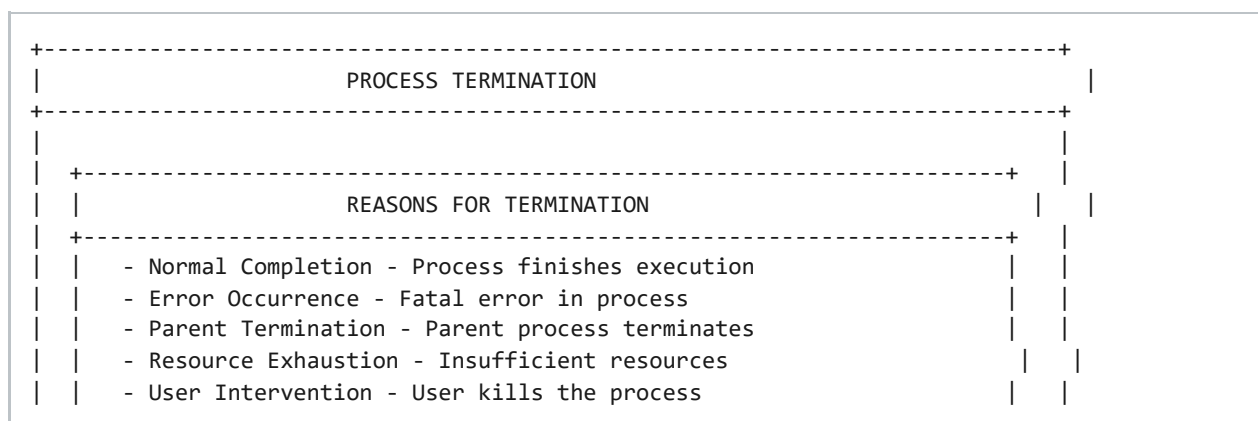
Function	Description
Context Switching	Saves and restores process state
Process Management	Tracks process state transitions
Resource Management	Manages allocated resources
Security	Identifies process owner
Scheduling	Contains priority information

6. Process Creation & Termination

Process Creation



Process Termination



- Time Exceeded - Process exceeds time limit
- Memory Exceeded - Process exceeds memory limit
- Protection Error - Illegal memory access

STEPS IN PROCESS TERMINATION

1. Identify the process to terminate
2. Remove process from all queues
3. Deallocate all resources
4. Remove PCB and free memory
5. Notify parent process (if any)

Process Hierarchy

PROCESS HIERARCHY

INIT Process
(PID = 1)

Process A
(PID=2)

Process B
(PID=3)

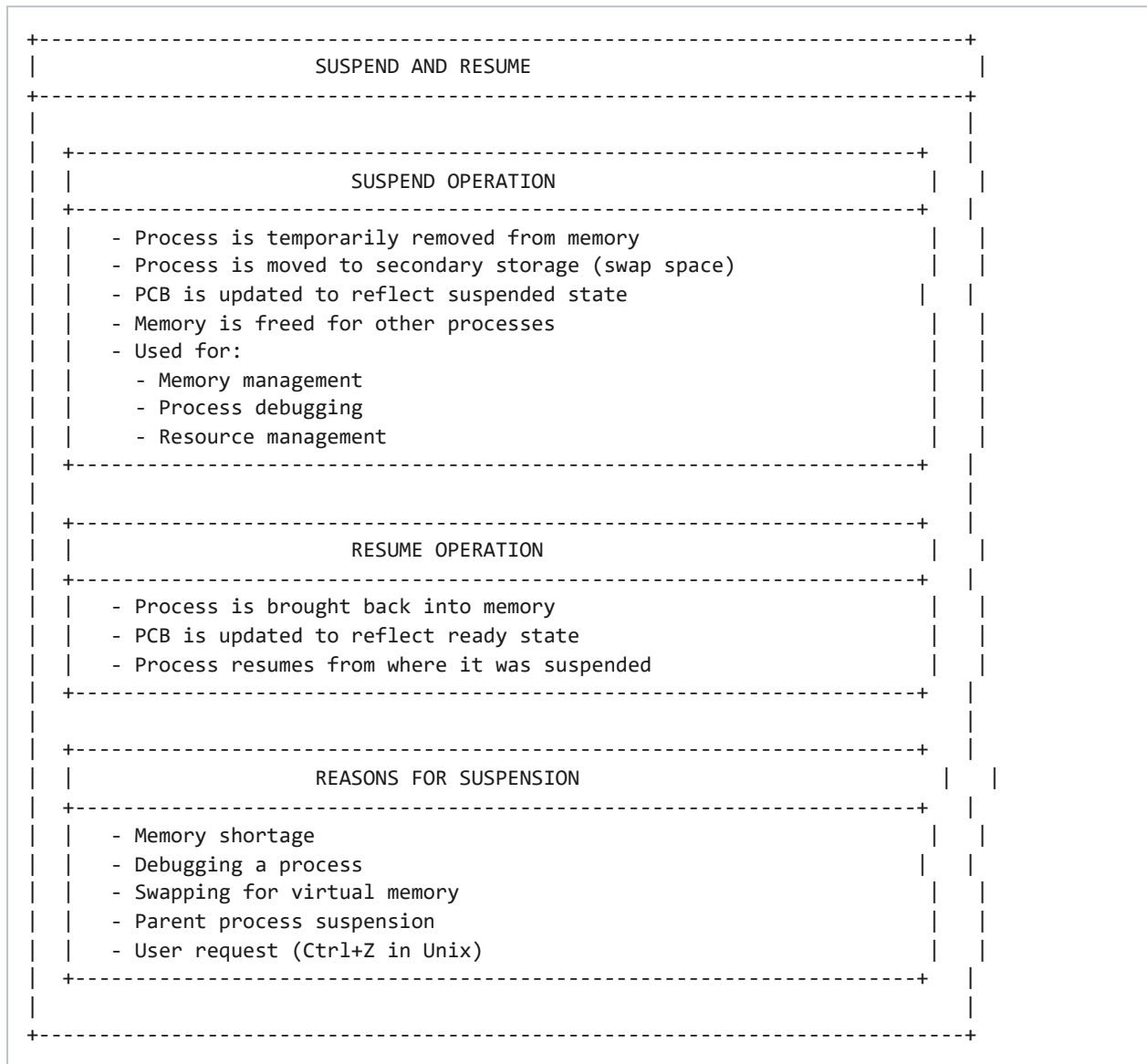
Process C
(PID=4)

Child 1
(PID=5)

Child 2
(PID=6)

- Each process has a parent (except init)
- Parent can create multiple children
- Process forms a tree structure

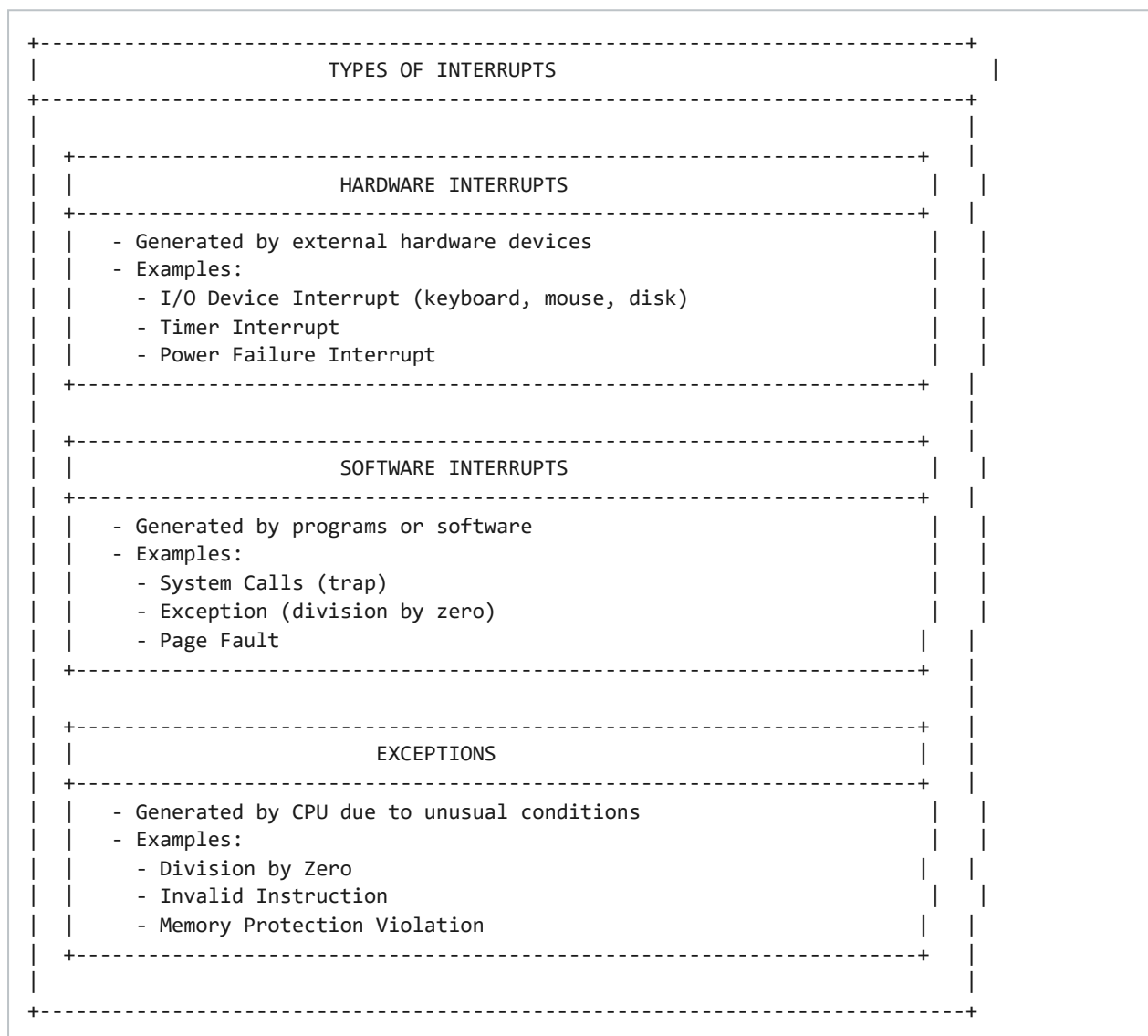
6. Suspend and Resume Operations



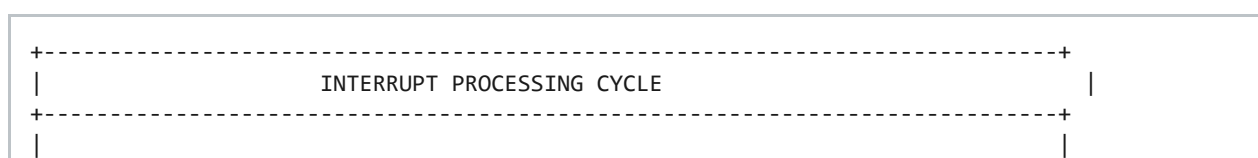
7. Interrupts

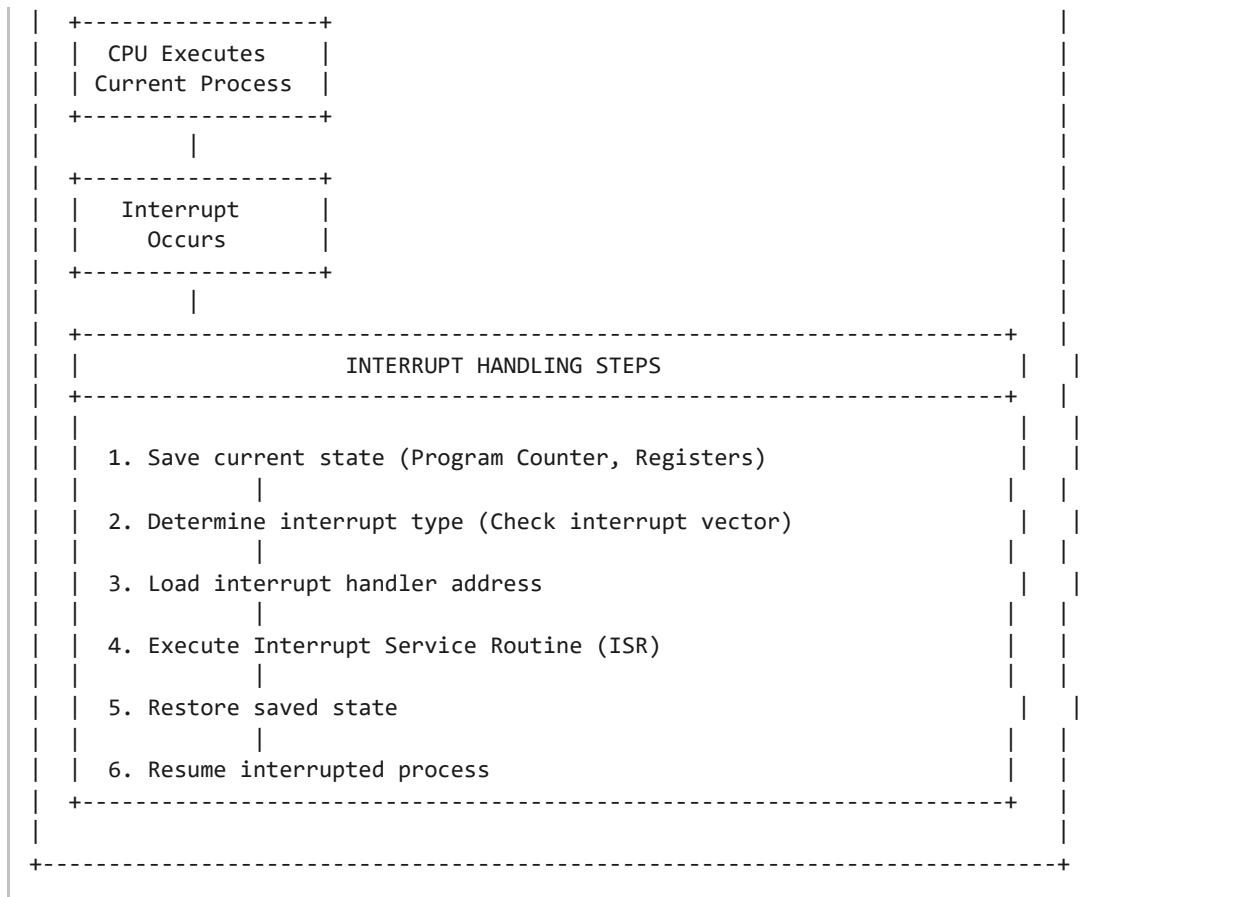
An **Interrupt** is a signal sent to the CPU that temporarily stops the current program execution and transfers control to a special program called the **Interrupt Handler**.

Types of Interrupts



Interrupt Processing Cycle





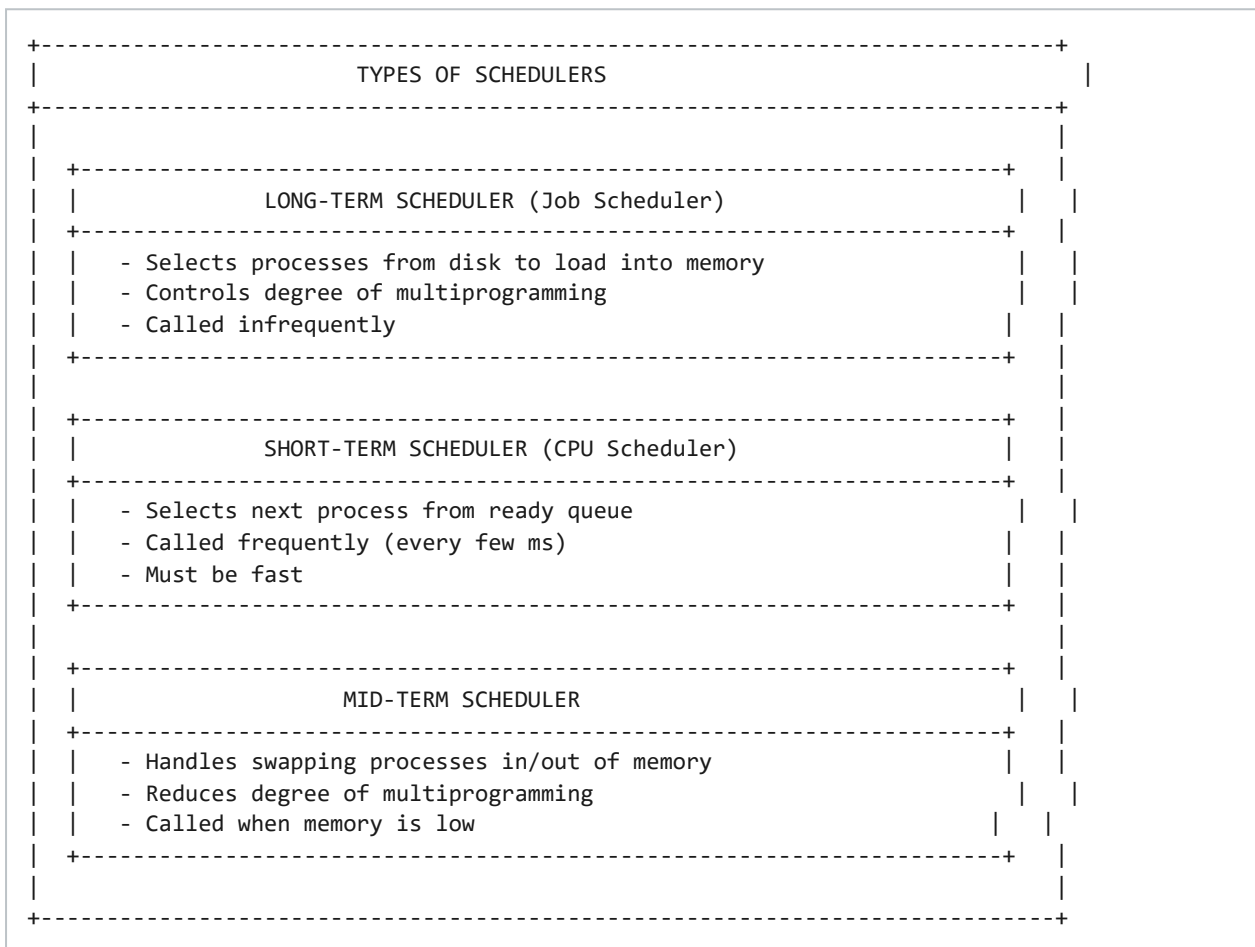
Interrupt Vector Table

INTERRUPT VECTOR TABLE		
Interrupt Number	Priority	ISR Address
0	Highest	Address 1 (Divide by Zero)
1	High	Address 2 (Timer)
2	High	Address 3 (Keyboard)
3	Medium	Address 4 (Mouse)
4	Medium	Address 5 (Disk I/O)
5	Low	Address 6 (Network)
...	...	...

7. Scheduling

Scheduling is the process of selecting which process from the ready queue will be executed next by the CPU.

Types of Schedulers



Scheduling Criteria

Criteria	Description
CPU Utilization	Keep CPU as busy as possible (40-90%)
Throughput	Number of processes completed per unit time

Turnaround Time	Time from submission to completion
Waiting Time	Total time process spends in ready queue
Response Time	Time from submission to first response
Fairness	Ensure all processes get fair CPU time
Efficiency	Minimize overhead and maximize throughput

8. CPU Scheduling Algorithms

8.1 First-Come-First-Served (FCFS) Scheduling

FCFS SCHEDULING		
- Non-preemptive - First process to arrive gets CPU first - Simple and easy to implement - Can cause Convoy Effect		
Example:		

Process	Burst Time	Arrival Time
P1	24	0
P2	3	1
P3	3	2
Gantt Chart:		
+-----+		
	P1	P2 P3
+-----+		
0	24	27 30
Average Waiting Time = $(0 + (24-1) + (27-2)) / 3 = 16\text{ms}$		

8.2 Shortest Job First (SJF) Scheduling

SJF SCHEDULING	
- Non-preemptive (or preemptive - SRTF) - Process with shortest burst time gets CPU first - Optimal average waiting time - Problem: Starvation possible	
Example (Non-preemptive):	

Process	Burst Time
P1	6
P2	8
P3	7
P4	3
Gantt Chart:	

P4	P1	P3	P2
0	3	9	16
24			

Average Waiting Time = $(0 + 3 + 9 + 16) / 4 = 7\text{ms}$

8.3 Priority Scheduling

PRIORITY SCHEDULING		
- Process with highest priority gets CPU first - Can be preemptive or non-preemptive - Problem: Starvation (low priority processes) - Solution: Aging (increase priority over time)		
Example:		
Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2
Gantt Chart:		
<pre> +-----+ P2 P5 P1 P3 P4 +-----+ 0 1 6 16 18 19 </pre>		
Average Waiting Time = $(0 + 1 + 6 + 16 + 18) / 5 = 8.2\text{ms}$		

8.4 Round Robin (RR) Scheduling

```

+-----+
|                                     ROUND ROBIN SCHEDULING                                     |
+-----+
|
| - Preemptive
| - Each process gets a fixed time quantum
| - Uses FIFO ready queue
| - Fair allocation of CPU
| - Performance depends on quantum size
|
| Example (Time Quantum = 4ms):
| -----
| Process   Burst Time

```

```

P1      24
P2      3
P3      3

```

Gantt Chart:

```

+-----+
| P1 | P2 | P3 | P1 | P1 | P1 | P1 |   |
+-----+
| 0  | 4  | 7  | 10 | 14 | 18 | 22 | 26 |

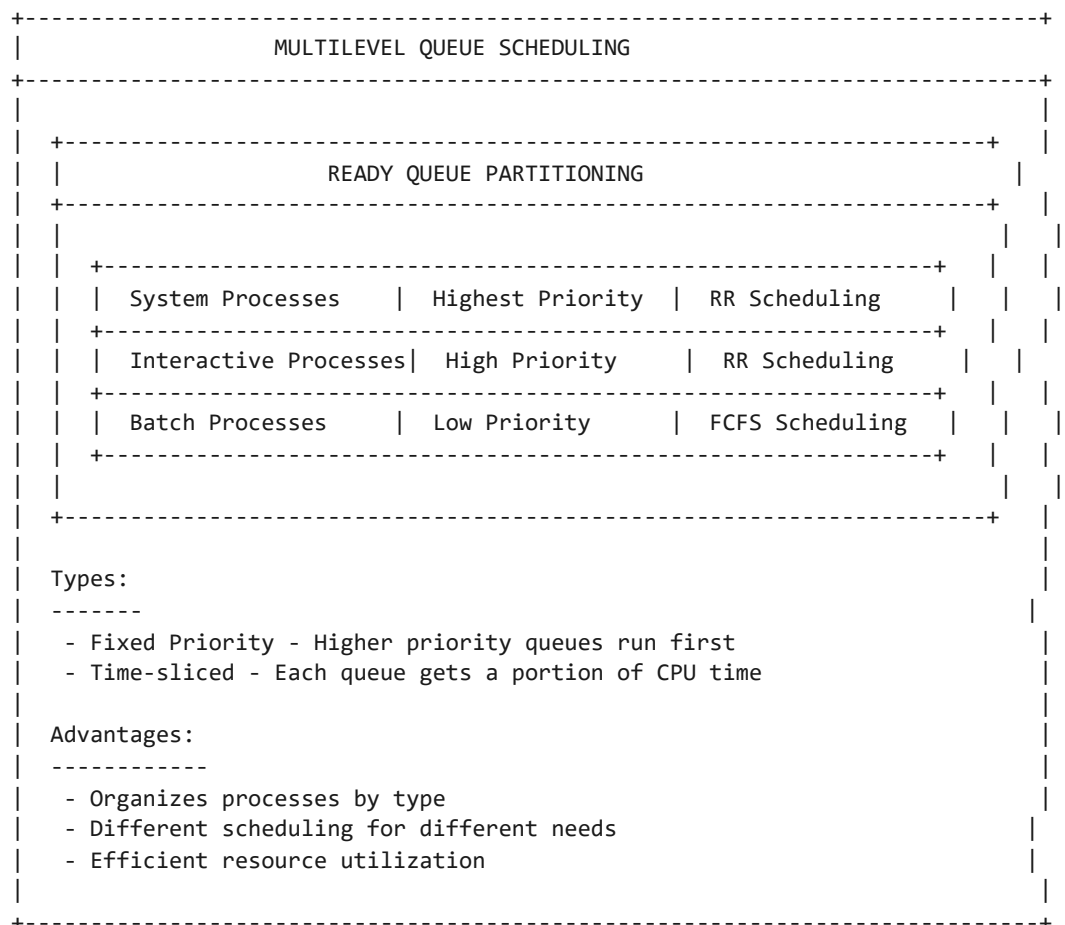
```

Average Waiting Time = $(6 + 4 + 7) / 3 = 5.67\text{ms}$

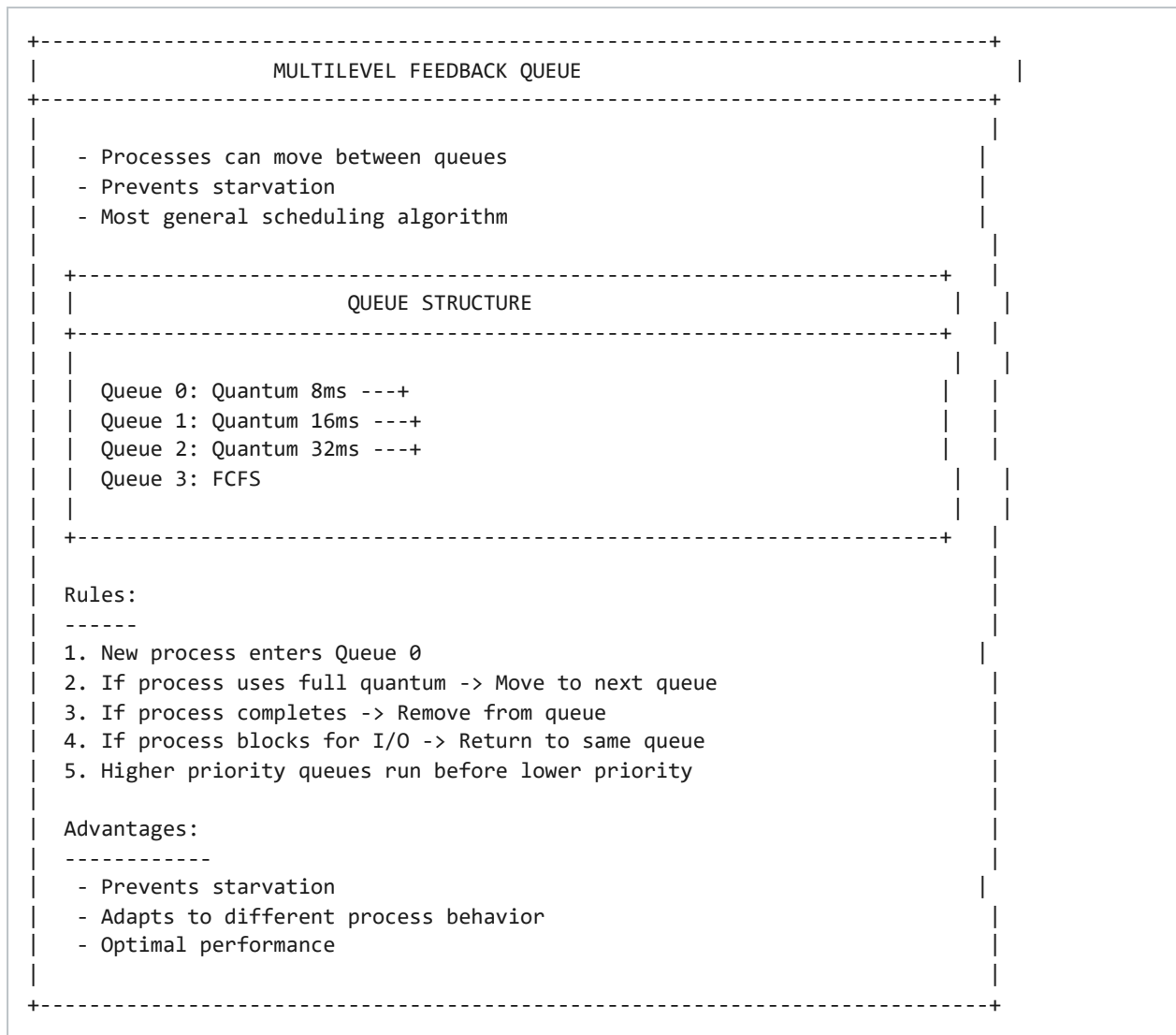
Effect of Quantum Size:

- Small Quantum -> More context switches -> More overhead
- Large Quantum -> Less context switches -> Better throughput
- Very Large Quantum -> Becomes FCFS
- Ideal Quantum: 10-100ms

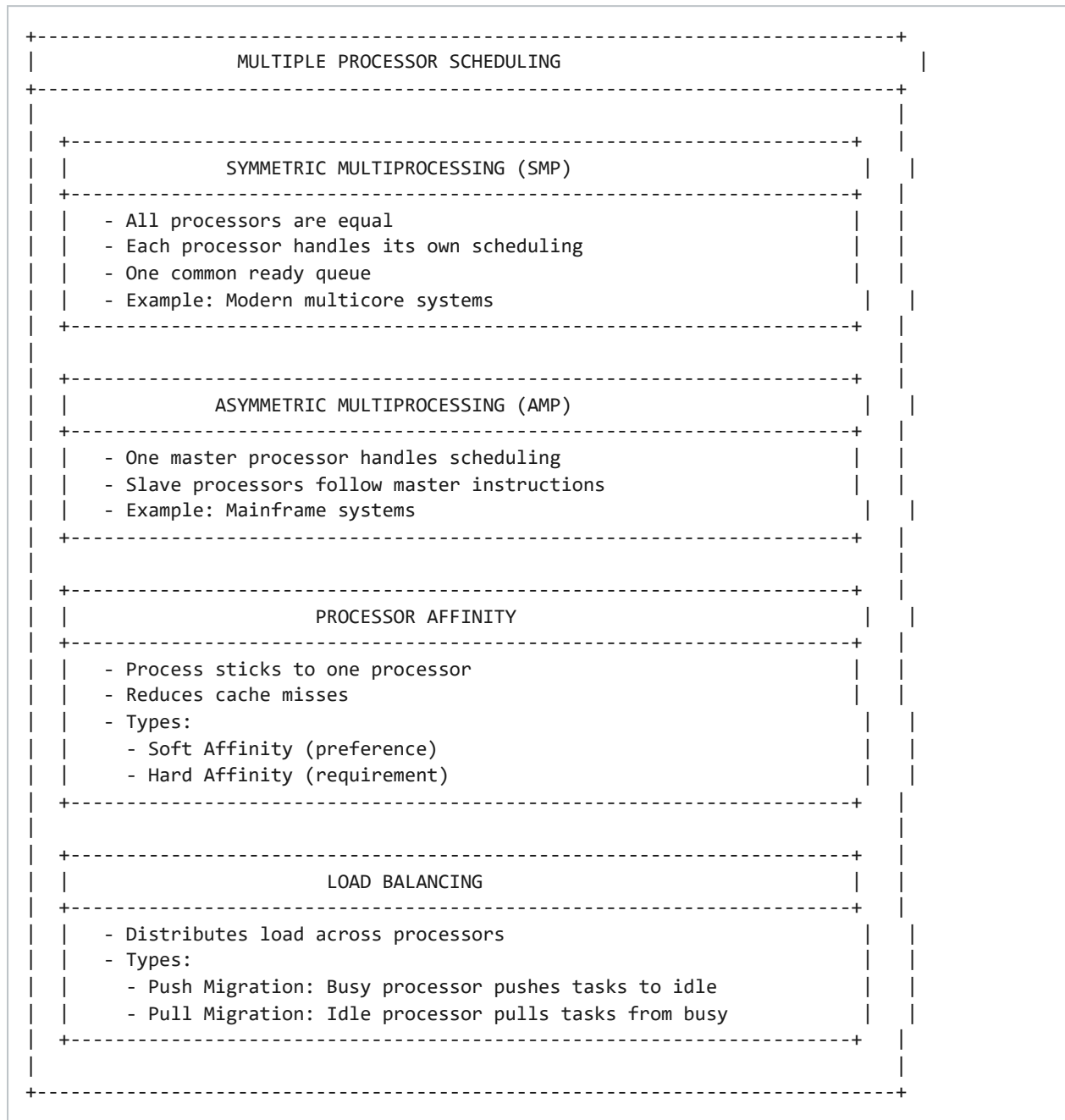
8.5 Multilevel Queue Scheduling



8.6 Multilevel Feedback Queue Scheduling



9. Multiple Processor Scheduling



10. Process Synchronization

10.1 Critical Section Problem

The **Critical Section Problem** deals with ensuring that when one process is executing in its critical section (code accessing shared resources), no other process can enter its critical section.

Requirements for Solution

Requirement	Description
Mutual Exclusion	Only one process in critical section at a time
Progress	Only processes not in critical section can decide who enters next
Bounded Waiting	A process cannot wait indefinitely to enter

10.2 Semaphores

A **Semaphore** is a synchronization primitive used to control access to shared resources. It is an integer variable accessed only through **P (wait)** and **V (signal)** operations.

- **P(S)** - Decrements semaphore; if negative, process blocks
- **V(S)** - Increments semaphore; wakes up a blocked process

10.3 Mutex

A **Mutex** (Mutual Exclusion) is a binary semaphore used to ensure only one process enters the critical section at a time.

10.4 Monitors

A **Monitor** is a high-level synchronization construct that encapsulates shared variables, procedures, and synchronization logic into a single unit. Only one process can execute inside a monitor at a time.

11. Classical Synchronization Problems

11.1 Producer-Consumer Problem

The Producer-Consumer problem involves two processes sharing a buffer:

- Producer produces items and puts them in buffer
- Consumer takes items from buffer and consumes them
- Producer cannot produce when buffer is full
- Consumer cannot consume when buffer is empty

Solution using Semaphores

```
semaphore mutex = 1; // Mutual exclusion
semaphore empty = n; // Empty slots
semaphore full = 0;  // Filled slots

// Producer
void producer() {
    while (true) {
        produce_item();
        wait(empty);
        wait(mutex);
        put_item_in_buffer();
        signal(mutex);
        signal(full);
    }
}

// Consumer
void consumer() {
    while (true) {
        wait(full);
        wait(mutex);
        remove_item_from_buffer();
        signal(mutex);
        signal(empty);
        consume_item();
    }
}
```



```
}  
}
```

11.2 Readers-Writers Problem

Multiple readers can read simultaneously, but writers need exclusive access. Solutions prioritize either readers or writers.

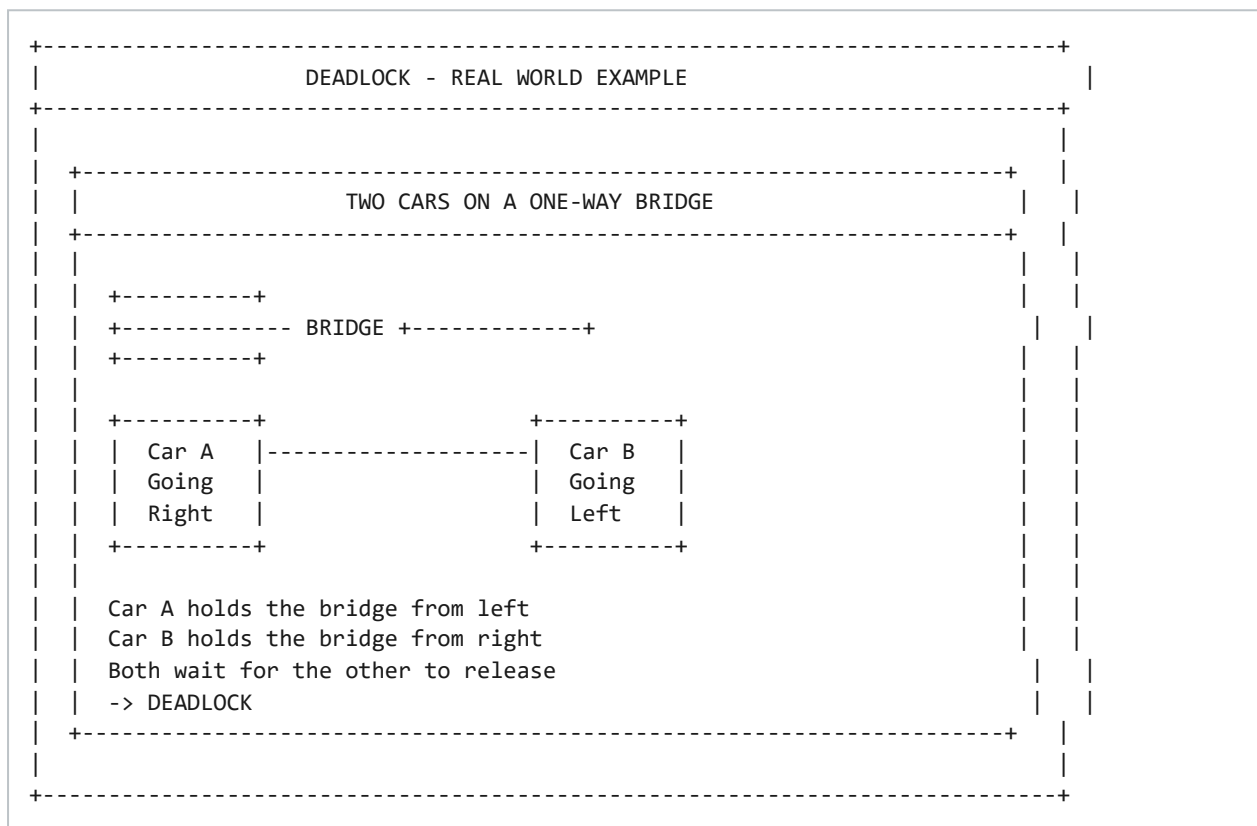
11.3 Dining Philosophers Problem

```
+-----+  
|          DINING PHILOSOPHER'S PROBLEM          |  
+-----+  
|  
| Problem Description:  
| - Five philosophers sit around a circular table  
| - Each philosopher needs two chopsticks to eat  
| - Each philosopher alternates between thinking and eating  
| - One chopstick between each pair of philosophers  
|  
| Problem: If all philosophers pick up their left chopstick  
| simultaneously, deadlock occurs.  
|  
| Solution using Semaphores:  
| -----  
| semaphore chopstick[5] = {1, 1, 1, 1, 1};  
|  
| void philosopher(int i) {  
|     while (true) {  
|         think();  
|         wait(chopstick[i]);          // Pick left  
|         wait(chopstick[(i+1)%5]);    // Pick right  
|         eat();  
|         signal(chopstick[i]);        // Release left  
|         signal(chopstick[(i+1)%5]); // Release right  
|     }  
| }  
|  
| Additional Solutions to Avoid Deadlock:  
| 1. Allow only 4 philosophers at a time  
| 2. Asymmetric approach (odd picks left first, even picks right first)  
| 3. Use a mutex lock  
| 4. Philosopher picks up both chopsticks at once  
|  
+-----+
```

12. Deadlock

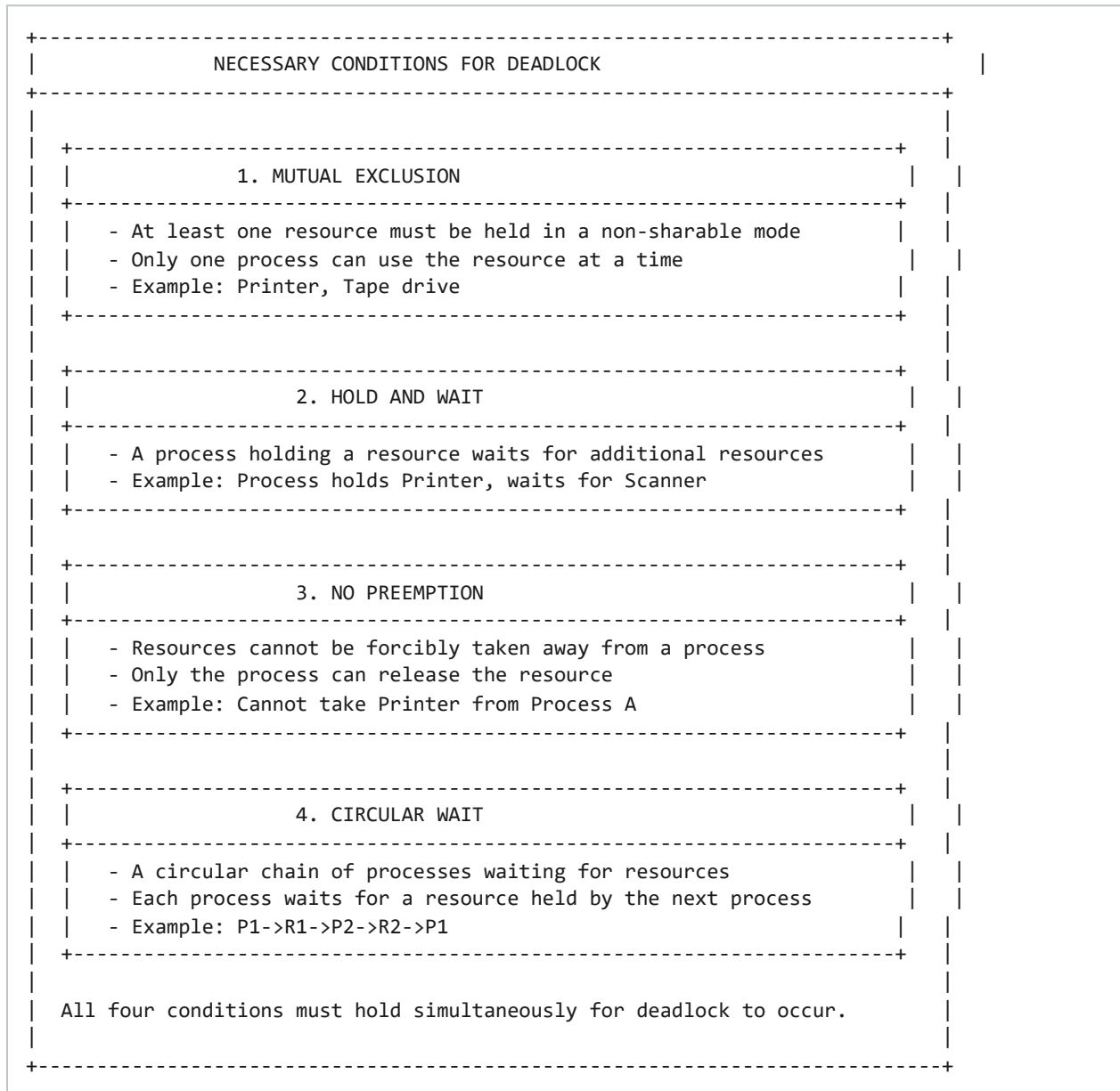
A **Deadlock** is a situation where two or more processes are **waiting indefinitely** for resources held by each other. Each process holds a resource and waits for another resource held by another process, creating a **circular wait**.

Real-World Example

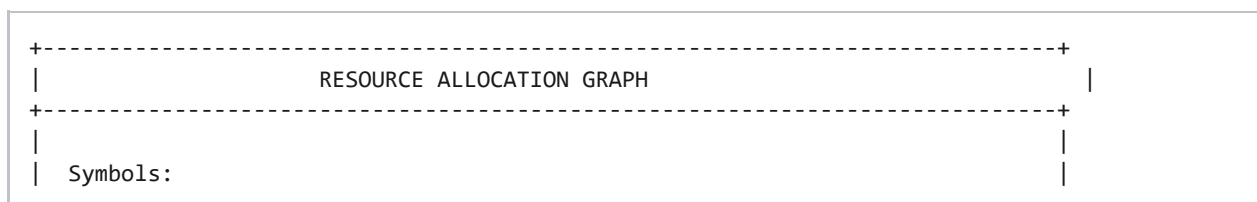


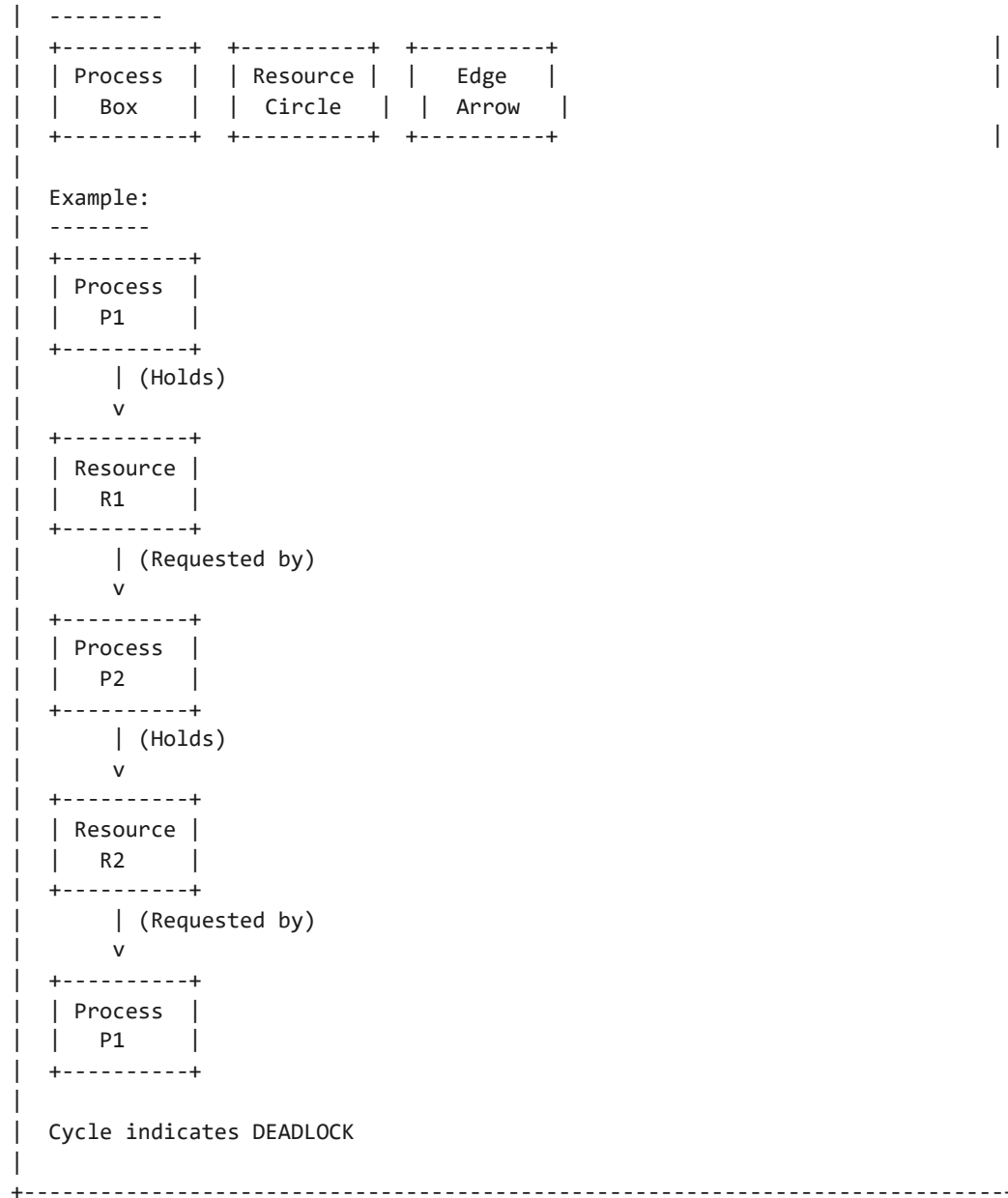
11. Deadlock Characterization

Four Necessary Conditions for Deadlock

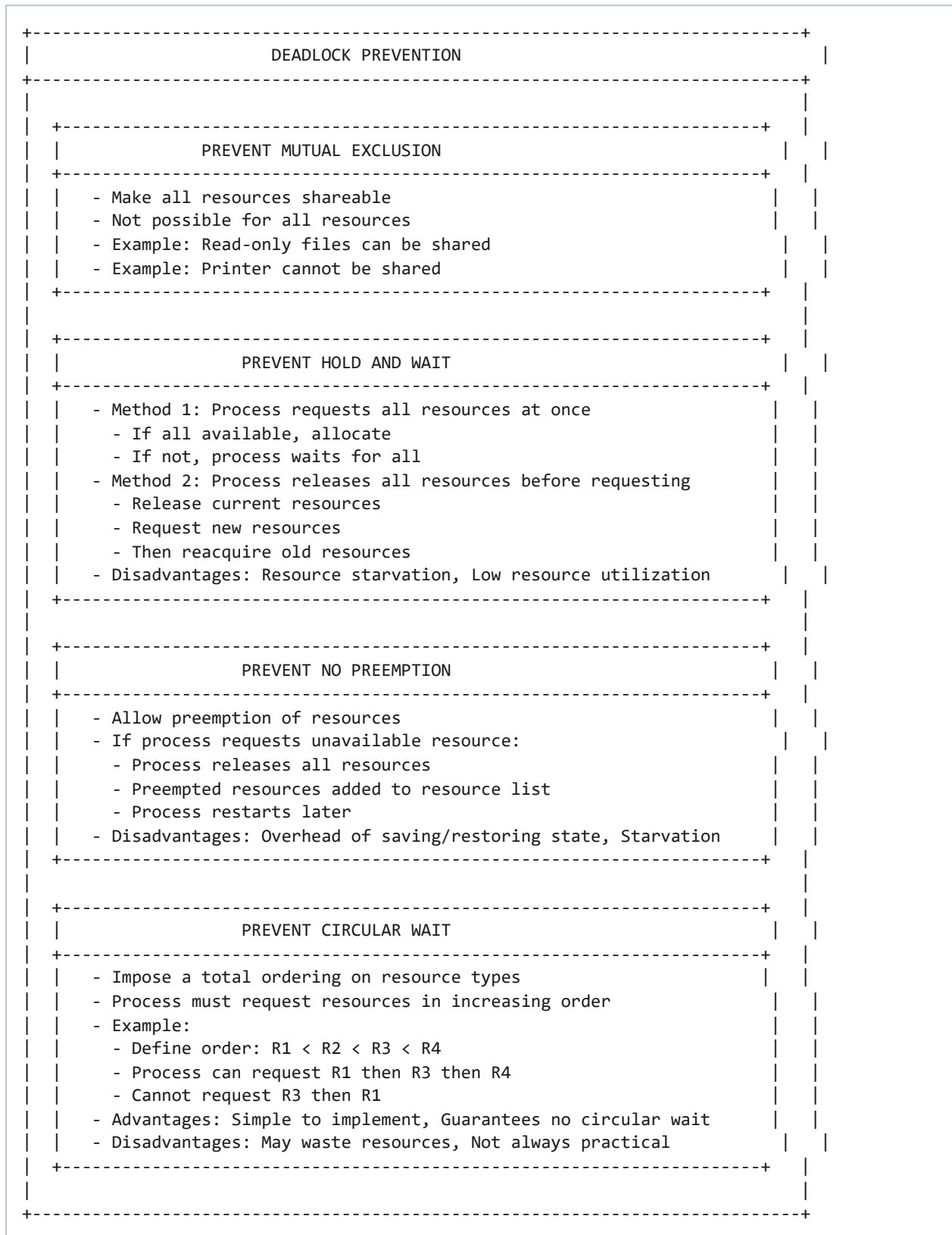


Resource Allocation Graph





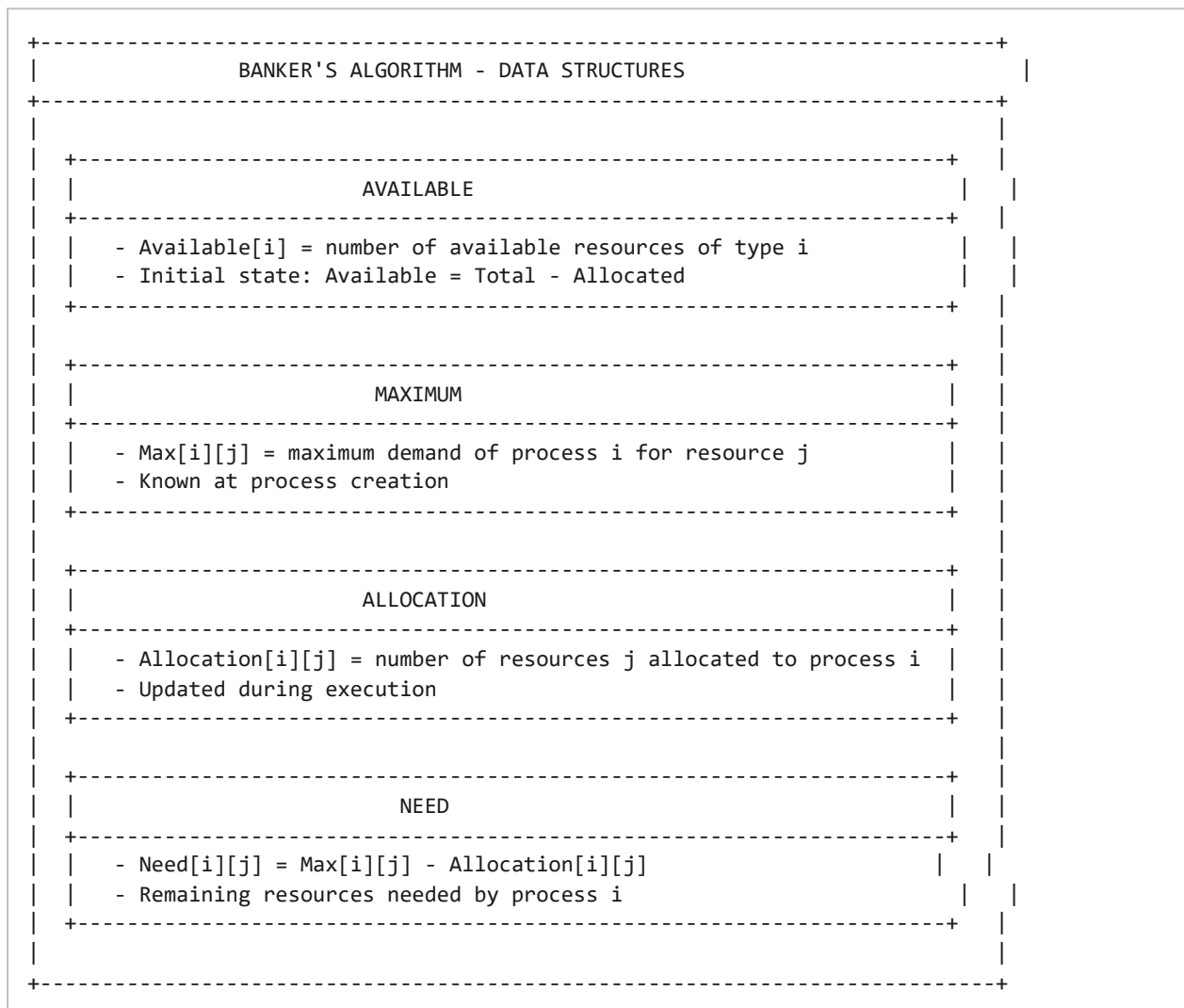
12. Deadlock Prevention



13. Deadlock Avoidance - Banker's Algorithm

The **Banker's Algorithm** is a **deadlock avoidance** algorithm that ensures the system never enters an unsafe state.

Data Structures



Banker's Algorithm - Complete Example

Problem: Consider the following system snapshot:

Process

Allocation

Max

Available

	A	B	C	A	B	C	A	B	C
P0	0	0	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

Step 1: Calculate Need Matrix

Process	Need (A, B, C) = Max - Allocation
P0	$(7, 5, 3) - (0, 0, 0) = (7, 5, 3)$
P1	$(3, 2, 2) - (2, 0, 0) = (1, 2, 2)$
P2	$(9, 0, 2) - (3, 0, 2) = (6, 0, 0)$
P3	$(2, 2, 2) - (2, 1, 1) = (0, 1, 1)$
P4	$(4, 3, 3) - (0, 0, 2) = (4, 3, 1)$

Step 2: Find Safe Sequence

Available = (3, 3, 2)

1. P3: $(0, 1, 1) \leq (3, 3, 2) \rightarrow$ Available = (5, 4, 3)
2. P1: $(1, 2, 2) \leq (5, 4, 3) \rightarrow$ Available = (7, 4, 3)
3. P4: $(4, 3, 1) \leq (7, 4, 3) \rightarrow$ Available = (7, 4, 5)
4. P2: $(6, 0, 0) \leq (7, 4, 5) \rightarrow$ Available = (10, 4, 7)
5. P0: $(7, 5, 3) \leq (10, 4, 7) \rightarrow$ Available = (10, 4, 7)

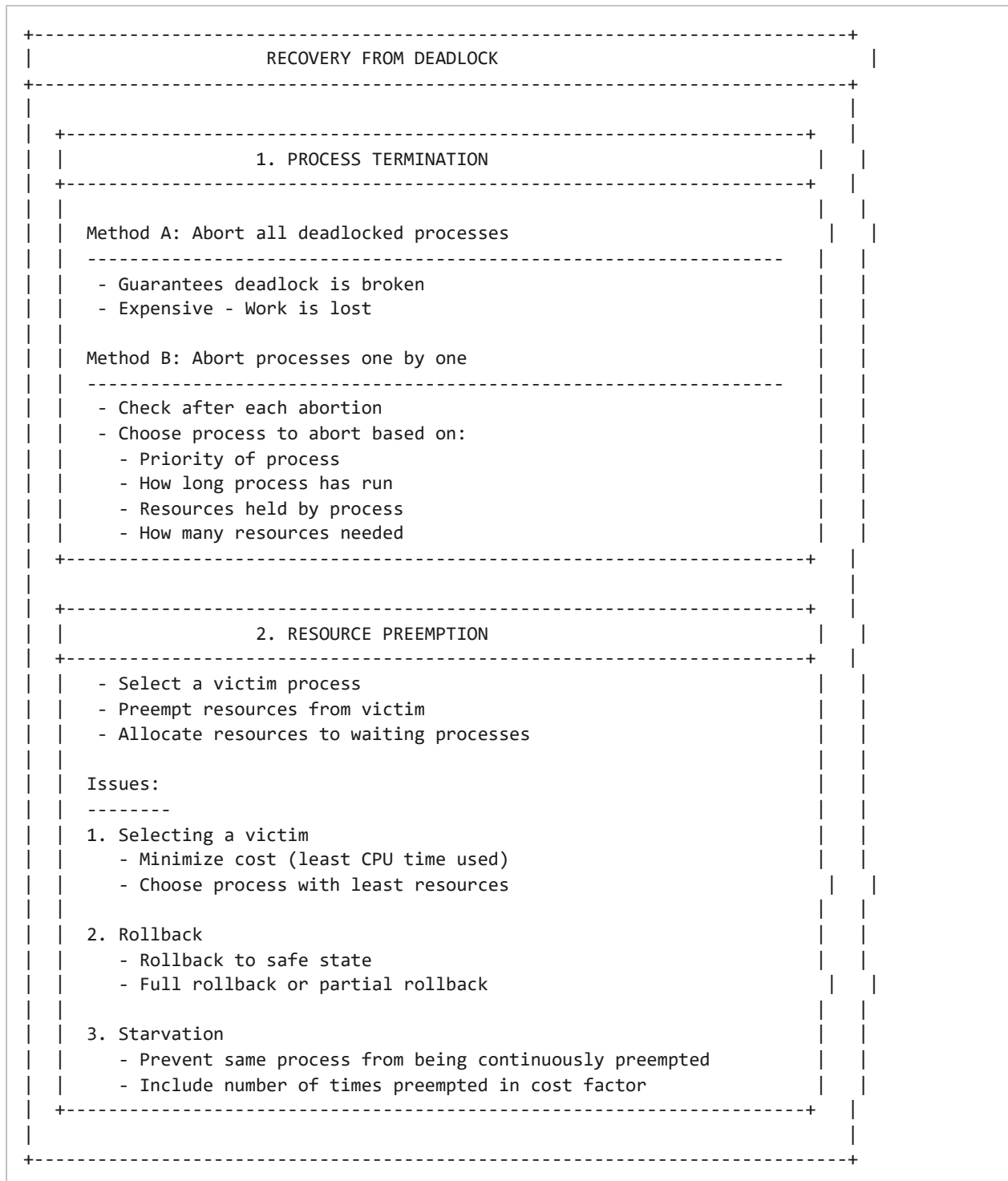
Safe Sequence: P3 -> P1 -> P4 -> P2 -> P0

System is in a **safe state**.

13. Deadlock Detection

+-----+ DEADLOCK DETECTION +-----+		
+-----+ SINGLE INSTANCE OF EACH RESOURCE TYPE +-----+		
- Use Resource Allocation Graph - Detect cycles in the graph - Cycle = Deadlock +-----+		
+-----+ MULTIPLE INSTANCES OF RESOURCES +-----+		
- Use Banker's Algorithm (Safety Algorithm) - If no process can finish -> Deadlock +-----+		
Detection Algorithm (For multiple instances): +-----+		
1. Initialize Work = Available 2. Find process with Need <= Work 3. If found, add allocation to Work 4. Repeat until no process can be processed 5. If some processes remain -> Deadlock +-----+		

14. Recovery from Deadlock



15. Questions & Answers

SECTION A: Very Short Answer Questions (2-3 Marks)

Q1. Define a Process. What are the different states of a process?

Answer: A **Process** is a **program in execution**. It is an active entity that includes the program code, program counter, registers, stack, and data section.

Process States: New, Ready, Running, Waiting/Blocked, Terminated

Q2. What is a Process Control Block (PCB)? What information does it contain?

Answer: A **Process Control Block (PCB)** is a **data structure** maintained by the operating system for every process. Contents: Process ID, Process State, Program Counter, CPU Registers, Memory Management Info, Accounting Info, I/O Status Info.

Q3. Define Throughput and Turnaround Time.

Answer: Throughput - The number of processes completed per unit time. **Turnaround Time** - The total time taken from the submission of a process to its completion.

Q4. What is the difference between Preemptive and Non-Preemptive Scheduling?

Preemptive Scheduling	Non-Preemptive Scheduling
CPU can be taken away from a process	CPU is held until process completes
High overhead due to frequent context switching	Low overhead
Suitable for time-sharing systems	Suitable for batch systems

Process can be interrupted	Process runs to completion
Example: Round Robin	Example: FCFS

Q5. Define Deadlock. What are the four necessary conditions?

Answer: A **Deadlock** is a situation where two or more processes are waiting indefinitely for resources held by each other. Four conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

Q6. What is a Semaphore?

Answer: A **Semaphore** is a synchronization primitive used to control access to shared resources. It is an integer variable accessed only through **P (wait)** and **V (signal)** operations.

Q7. What is the difference between Deadlock and Starvation?

Deadlock	Starvation
All processes are blocked and none can proceed	A process is indefinitely delayed but other processes progress
Deadlock situation	Livelock situation

Q8. What is the Critical Section Problem?

Answer: The **Critical Section Problem** deals with ensuring that when one process is executing in its critical section, no other process can enter its critical section. Requirements: Mutual Exclusion, Progress, Bounded Waiting.

Q9. Explain Response Time and Waiting Time.

Answer: Response Time - Time from submission of a request to the first response.

Waiting Time - Total time a process spends waiting in the ready queue.

Q10. What is Context Switching? Why is it needed?

Answer: Context Switching is the process of saving the state of a currently executing process and loading the saved state of a new process to the CPU. It allows multiple processes to share CPU, enables multitasking, and is essential for preemptive scheduling.

SECTION B: Short Answer Questions (5 Marks)

Q11. Explain the Process State Transition Diagram.

Answer: The process state transition diagram shows how a process moves between states: New -> Ready (admitted), Ready -> Running (dispatched), Running -> Ready (preempted), Running -> Waiting (I/O request), Waiting -> Ready (I/O complete), Running -> Terminated (exit).

Q12. Explain any THREE CPU Scheduling Algorithms with examples.

1. FCFS: Non-preemptive, first process to arrive gets CPU first. Example: P1(24), P2(3), P3(3) -> Avg WT = 16ms.

2. SJF: Process with smallest burst time gets CPU first. Example: P4(3), P1(6), P3(7), P2(8) -> Avg WT = 7ms.

3. Round Robin: Preemptive with time quantum. Example (Quantum=4ms): P1(24), P2(3), P3(3) -> Avg WT = 5.67ms.

Q13. Explain the four necessary conditions for deadlock with examples.

Answer: 1. **Mutual Exclusion** - Resources cannot be shared (e.g., Printer). 2. **Hold and Wait** - Process holds resources while waiting for more. 3. **No Preemption** - Resources cannot be forcibly taken away. 4. **Circular Wait** - Circular chain of waiting processes.

Q14. Difference between Deadlock Prevention and Deadlock Avoidance

Aspect	Deadlock Prevention	Deadlock Avoidance
Approach	Eliminate one necessary condition	Ensure system never enters unsafe state
Resource Allocation	Conservative	Dynamic
Examples	Resource ordering, Preemption	Banker's Algorithm

Overhead	Low	High
Flexibility	Less flexible	More flexible
When applied	Before execution	During execution
Implementation	Simple	Complex

Q15. Explain the Banker's Algorithm with a suitable example.

Answer: The Banker's Algorithm is a deadlock avoidance algorithm. It checks if allocating resources to a process will leave the system in a safe state. (See detailed example in Section 12 above.)

Q16. Explain the Producer-Consumer Problem using Semaphores.

Answer: The Producer-Consumer problem involves two processes sharing a buffer. Producer produces items and puts them in buffer; Consumer takes items from buffer. Solution uses three semaphores: `mutex` (mutual exclusion), `empty` (empty slots), `full` (filled slots).

Q17. Explain the Dining Philosopher's Problem.

Answer: Five philosophers sit around a table, each needs two chopsticks to eat. If all pick up left chopstick simultaneously, deadlock occurs. Solutions: Allow only 4 philosophers, asymmetric approach, use mutex lock.

Q18. Explain Deadlock Detection and Recovery methods.

Answer: Detection: Single instance - Resource Allocation Graph (cycle = deadlock). Multiple instances - Detection Algorithm. **Recovery:** Process Termination (abort all or one by one) and Resource Preemption (select victim, rollback, prevent starvation).

SECTION C: Long Answer Questions (10-15 Marks)

Q19. Explain in detail all CPU Scheduling Algorithms with examples and Gantt charts.

Answer: (See detailed explanation in Section 8 of these notes.) Algorithms: FCFS, SJF/SRTF, Priority, Round Robin, Multilevel Queue, Multilevel Feedback Queue.

Q20. Explain the Banker's Algorithm for Deadlock Avoidance with a detailed numerical example.

Answer: (See detailed numerical example in Section 12 of these notes.) Steps: 1. Calculate Need = Max - Allocation. 2. Set Work = Available, Finish = False. 3. Find process with Need $\leq$ Work and Finish = False. 4. If found, add Allocation to Work, set Finish = True. 5. Repeat. 6. If all Finish = True $\rightarrow$ Safe state, else $\rightarrow$ Unsafe state.

Q21. Write a detailed note on Deadlock: (a) Necessary Conditions (b) Prevention (c) Avoidance (d) Detection and Recovery

Answer: (a) **Necessary Conditions:** Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait. (b) **Prevention:** Prevent any one of the four conditions. (c) **Avoidance:** Banker's Algorithm, Resource Allocation Graph. (d) **Detection:** Single Instance - Resource Allocation Graph; Multiple Instances - Detection Algorithm. **Recovery:** Process Termination, Resource Preemption.

Q22. Explain Process Synchronization and the Critical Section Problem.

Answer: **Process Synchronization** is the coordination of multiple processes accessing shared resources. **Critical Section Problem:** Code section where shared resources are accessed. Only one process can execute critical section at a time. Requirements: Mutual Exclusion, Progress, Bounded Waiting. Solutions: Semaphores, Mutex Locks, Monitors.

Important Formulas for Numerical Problems

Formula	Description
Turnaround Time = Completion Time - Arrival Time	Total time from submission to completion
Waiting Time = Turnaround Time - Burst Time	Time spent waiting in ready queue
Response Time = First CPU Time - Arrival Time	Time from submission to first CPU allocation
CPU Utilization = (CPU Time) / (Total Time) x 100%	Percentage of time CPU is busy
Throughput = Number of Processes / Total Time	Processes completed per unit time
Context Switch Overhead = $n \times (\text{Switch Time})$	Total time spent switching contexts

Quick Revision Points

Topic	Key Points
Process States	New, Ready, Running, Waiting, Terminated
PCB Contents	PID, State, PC, Registers, Memory Info, I/O Status
Scheduling Algorithms	FCFS, SJF (SRTF), Priority, RR, Multilevel Queue, MLFQ
Deadlock Conditions	Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait
Deadlock Handling	Prevention, Avoidance (Banker's), Detection, Recovery
Synchronization	Semaphores, Mutex, Monitors, Critical Section
Classic Problems	Producer-Consumer, Dining Philosophers, Readers-Writers